

HW1

Team nineteen

<https://cal-cs184-student.github.io/hw1-nineteen-writeup/>

The renderer turns SVG shapes into pixels and adds transforms, color blending, and textures. More samples helped with jagged edges, while bilinear filtering and mipmaps helped with blocky or noisy textures.

Task 1

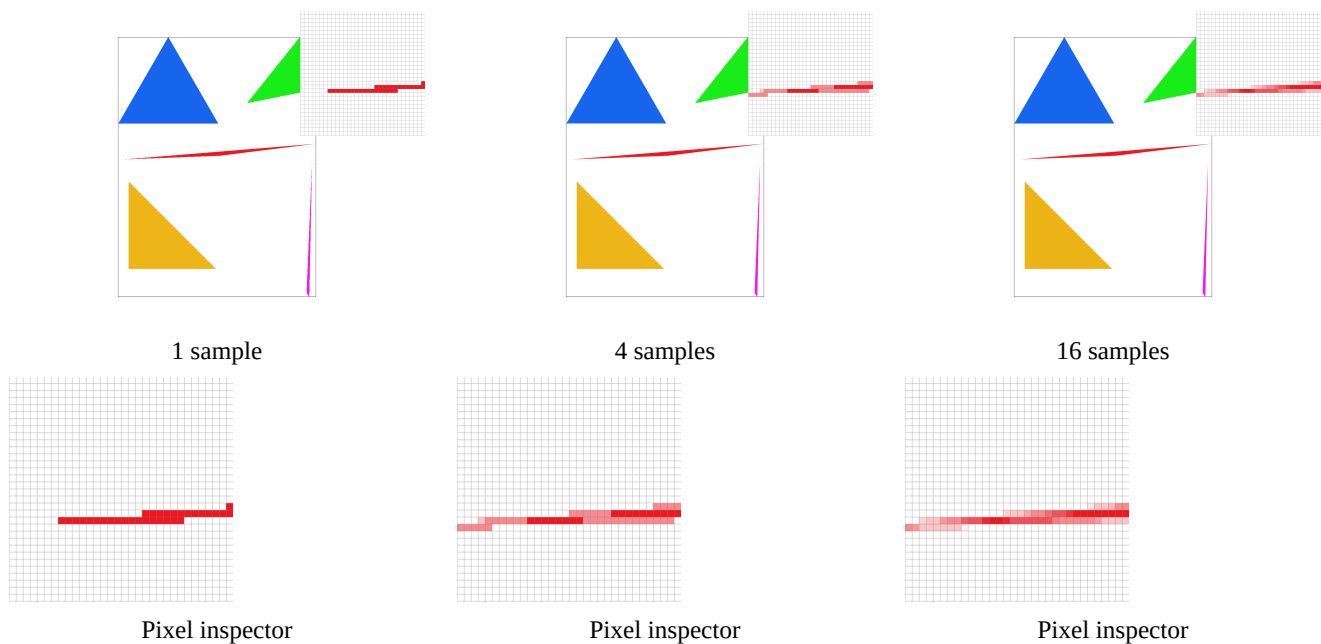
The loop starts with a box around the triangle, clipped to the screen, so it doesn't have to check every pixel on the screen. Inside that box, it checks pixel centers at $(x + 0.5, y + 0.5)$. Three edge tests tell whether the point is inside. If the results are all nonnegative or all nonpositive, the point gets drawn. Accepting either sign makes both vertex orders work, and accepting zero includes points on the edges. Triangles with zero area are skipped.

There are only three edge checks per sample, so the work grows with the number of samples in the bounding box. The picture labeled 1 sample below is also the result for this part.

Task 2

With supersampling, each pixel gets several sample points instead of one. For s samples, they form a $\sqrt{s}$ by $\sqrt{s}$ grid, with a point at the center of each small square. The colors go into `sample_buffer`, which has width * height * s entries. Each pixel's samples sit together in the buffer. Changing the window size or sample rate resizes it, and each new frame starts with white samples.

A triangle only fills the samples it covers. Points and lines fill all samples of their pixel. Once everything is drawn, `resolve_to_framebuffer` averages the samples to get one RGB color per pixel.

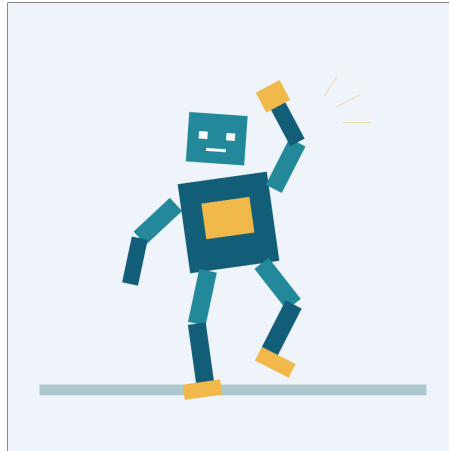


These are `basic/test4.svg` at the default view, with the inspector in the same spot. The red tip looks rough at 1 sample. At 4 and 16 samples, the edge has lighter red pixels and looks smoother. Those pixels are only partly covered, so averaging mixes the red with the white background.

Task 3

Translation moves the shape, scaling changes its size, and rotation turns it. The code uses 3 by 3 matrices for these. For rotation, the angle is converted from degrees to radians before using sine and cosine.

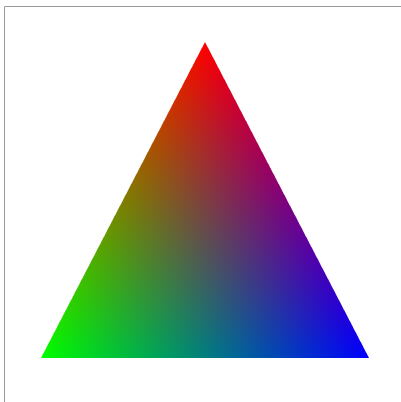
The idea for the robot was to have it wave while taking a step. Its body leans a little, with one arm up and one knee bent. The arm and leg parts are in nested groups, so moving a joint also moves the parts attached to it. For example, the forearm follows the shoulder because it is inside the upper-arm group. The file is `my_robot.svg`.



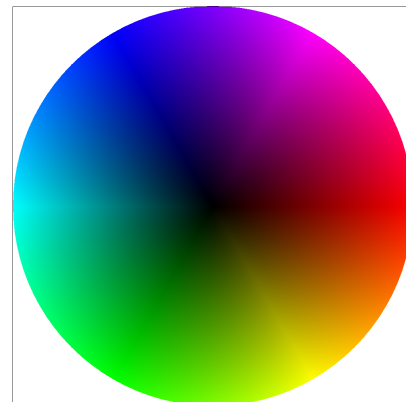
`my_robot.svg`, 16 samples per pixel.

Task 4

Barycentric coordinates give a point three weights, one for each vertex. They add up to 1. Each weight comes from the area of the opposite small triangle divided by the full triangle area. The color is then $a \cdot \text{colorA} + b \cdot \text{colorB} + c \cdot \text{colorC}$. At a vertex, only that vertex's color is used. Equal weights give an equal mix of the three colors.



RGB triangle, 1 sample



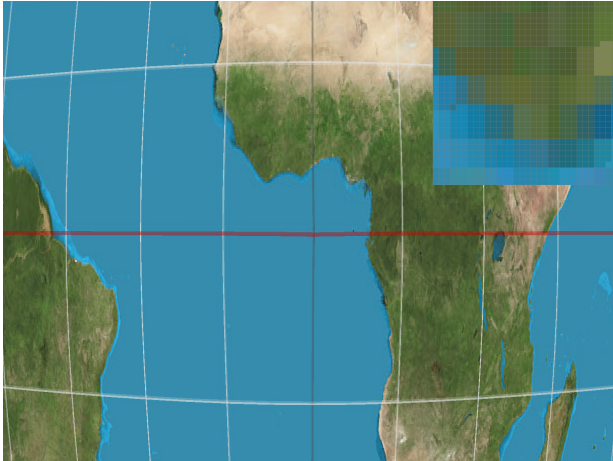
`basic/test7.svg`, default view, 1 sample

In the left picture, moving toward the top makes the color more red because the red weight gets larger. The bottom edge only mixes green and blue. The color wheel on the right uses the same blending across its triangles.

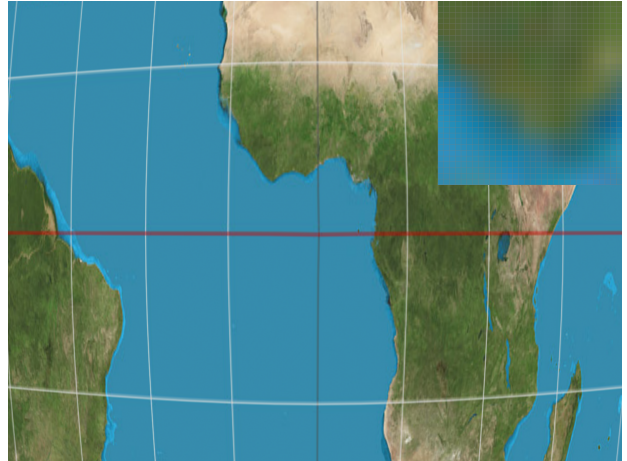
Task 5

For textures, the barycentric weights give the UV position of each sample. That position maps to $(u * (\text{width} - 1), v * (\text{height} - 1))$ in the texture. Indices at the edges are clamped so they don't go outside the image.

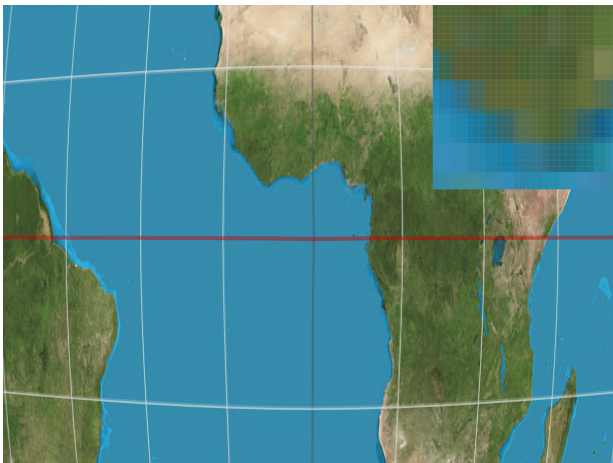
Nearest sampling just rounds to one texel and uses its color. Bilinear takes the four nearby texels and blends them based on how close the sample is to each one. It blends across each row first, then between the two rows.



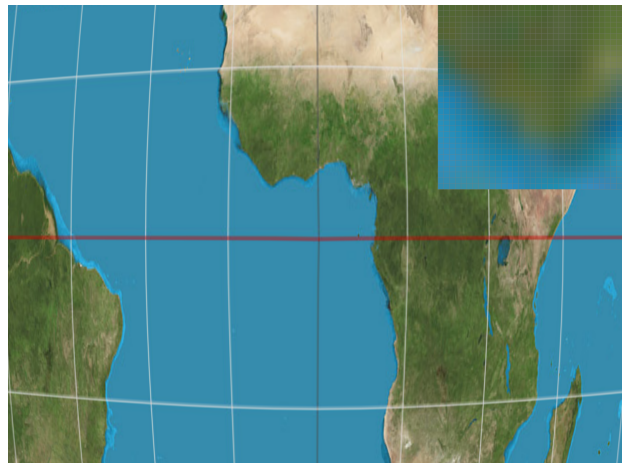
Nearest, 1 sample



Bilinear, 1 sample



Nearest, 16 samples



Bilinear, 16 samples

These show `texmap/test1.svg` with `L_ZERO` and the same zoom and inspector spot. Nearest at 1 sample looks blocky around the coast. Going to 16 samples softens those blocks. Bilinear already looks smoother at 1 sample, so going to 16 changes it less.

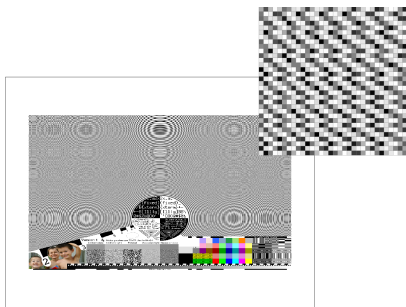
Zooming in makes the difference easier to see. One texel covers several screen pixels, so nearest repeats the same color over a block while bilinear blends into the next color. This matters most around sharp color changes. When zoomed far out, bilinear can still leave aliasing.

Task 6

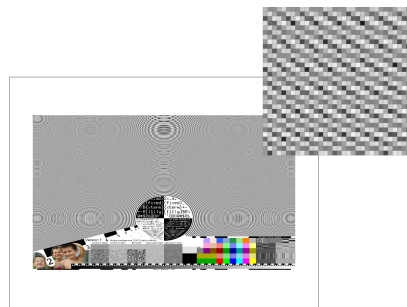
Mipmaps give the texture smaller, averaged versions to sample from. To pick a level, the code finds UV at (x,y) , $(x+1,y)$, and $(x,y+1)$, then subtracts the first UV from the other two. The u changes are multiplied by the original texture width and the v changes by its height. The longer change estimates how many texels fit in one screen pixel. Taking $\log_2$ gives the level, which is clamped to the levels that exist.

L_ZERO stays at level 0. L_NEAREST rounds to the closest level. L_LINEAR blends the two levels around the result. Nearest or bilinear pixel sampling works with any of these. Bilinear pixels with linear levels is trilinear filtering.

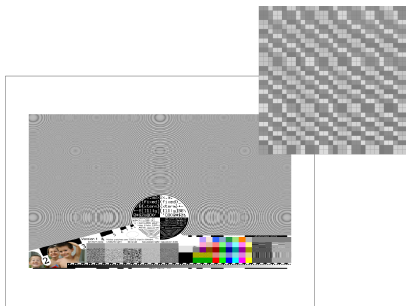
More screen samples help with edges, but take more time and buffer memory. Bilinear reads four texels instead of one, without needing another image, and helps when zoomed in. Mipmaps help when zoomed out, but a square power-of-two texture needs about one third more memory for its mip chain. Blending two levels takes more work than picking one, but avoids sudden level changes.



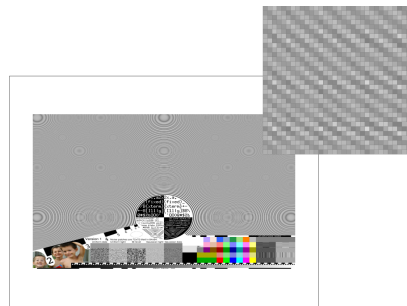
L_ZERO + P_NEAREST



L_ZERO + P_LINEAR



L_NEAREST + P_NEAREST



L_NEAREST + P_LINEAR

These use Zone plate boys at 1 sample and the same view. Level 0 has false patterns, and bilinear only reduces them a little. Using the nearest mip level removes more fine detail. Adding bilinear looks smoothest here. The texture covers about 1.8 texels per screen pixel, so the nearest mip level is 1.

The last mip levels initially came out black because the loop skipped them. Making the loop fill every level fixed this.

Some reference pixels still differ on thin edges and the color wheel. The starter also does not handle the partly transparent strokes in the lion and sphere.

Image: AlbertCahalan, Zone plate boys, CC BY-SA 3.0. The original PNG and filtered screenshots use this license. Codex helped with code, tests, and writing.

Texture source | CC BY-SA 3.0 license